



UNIVERSIDAD DE MÁLAGA

ESCUELA DE INGENIERÍAS INDUSTRIALES
LABORATORIO DE ROBÓTICA



INFORME DE PROYECTO

INGENIERÍA ELECTRÓNICA, ROBÓTICA Y
MECATRÓNICA

Curso 2025/26

Realizado por:
Arturo Ríos Pérez



Índice

1. Movimiento en línea recta con referencia de velocidad de 0.1 m/s durante 20 segundos, recorriendo 2 metros.	3
1.1. Estrategia de Control (Extrapolación del PID)	3
1.2. Implementación de la Odometría	4
1.3. Metodología de Calibración Experimental	4
1.4. Autoevaluación	5
2. Trazar una circunferencia de 0.4 m de radio y velocidad angular $w = \frac{2\pi}{10}$ rad/s.	6
2.1. Planteamiento del Problema y Cálculos Cinemáticos	6
2.2. Implementación en el Código de Control	6
2.3. Control de Duración de la Trayectoria	7
2.4. Autoevaluación	7
3. Recorrer 2 metros en línea recta con control de orientación.	8
3.1. Estrategia de Control: Odometría Diferencial	8
3.2. Implementación Matemática del Modelo	8
3.3. Mantenimiento del Rumbo (Control en Lazo Cerrado)	8
3.4. Autoevaluación	9
4. Implementar un algoritmo de seguimiento de trayectoria (Follow the carot)	10
4.1. Definición de la Trayectoria	10
4.2. Algoritmo de Control: Navegación Punto a Punto	10
4.3. Máquina de Estados de Navegación	11
4.4. Autoevaluación	12
5. Sistema de Seguridad Reactiva mediante Sensores de Proximidad	13
5.1. Integración y Adquisición de Datos	13
5.2. Estrategia de Lectura Secuencial	13
5.3. Lógica de Seguridad (Override de Control)	14
5.4. Autoevaluación	14
6. Implementación de la Tarea 5 con micro-ROS	15
6.1. Arquitectura del Sistema Distribuido	15
6.2. Configuración del Firmware ESP32	15
6.3. Nodo de Control en ROS 2: Follow Carot	16
6.4. Autoevaluación	16
7. Tareas Opcionales	17
7.1. Construcción, Diseño y Mejoras Electromecánicas	17
7.2. Implementación de Algoritmo de Seguimiento "FollowMe" mediante Sonar	18
7.2.1. Descripción del Procedimiento	18
7.2.2. Resultados y Conclusiones	18
7.2.3. Propuesta de Mejora	18
7.3. Interfaz Web de Teleoperación y Control Unificado	19



7.3.1. Modos de Control y Funcionalidades	19
7.3.2. Visualización Topológica, Mapeo SLAM y Navegación Táctil	20
7.4. Autoevaluación	21
8. Video donde ver las tareas	21

1. Movimiento en línea recta con referencia de velocidad de 0.1 m/s durante 20 segundos, recorriendo 2 metros.

1.1. Estrategia de Control (Extrapolación del PID)

Para lograr un movimiento rectilíneo preciso, se partió de la base del controlador PID desarrollado en la práctica anterior para un solo motor. Dado que el robot utiliza dos motores DC idénticos para la tracción diferencial, se asumió que la respuesta dinámica de ambos sistemas sería lo suficientemente similar como para compartir las mismas constantes de sintonización (K_p , K_i , K_d).

Para optimizar el código y evitar la duplicidad, se implementó una función de control genérica llamada `calcula_PID`. Esta función recibe por referencia las variables de estado de cada motor (error acumulado, error anterior, etc.), permitiendo calcular la señal PWM necesaria para cada rueda de forma independiente pero con la misma lógica de control.

```
1 double calcula_PID (double velocidad_input, double valor_buscado,
2   double &error_ant, double &error_acum, double &deriv_filt){
3   // Si la referencia es 0, reseteamos el PID para que no
4     acumule error y frene suave
5   if (valor_buscado == 0 && abs(velocidad_input) < 10) {
6     error_acum = 0;
7     return 0;
8   }
9   double error = valor_buscado - velocidad_input;
10  double derivada_raw = (error - error_ant) / tiempo_muestreo;
11  deriv_filt = alpha_derivada * deriv_filt + (1.0 -
12    alpha_derivada) * derivada_raw;
13  error_acum += error * tiempo_muestreo;
14
15  // Saturation del error acumulado (Anti-windup)
16  if (error_acum > 4000) error_acum = 4000;
17  if (error_acum < -4000) error_acum = -4000;
18
19  double pwm = Kp * error + Ki * error_acum + Kd * deriv_filt;
20  error_ant = error;
21
22  // Saturation del PWM
23  if (pwm > 1023) pwm = 1023;
24  else if (pwm < -1023) pwm = -1023;
25
26  return pwm;
27 }
```

Función Genérica PID

1.2. Implementación de la Odometría

El objetivo de avanzar exactamente 2 metros se gestiona mediante la lectura de los encoders en la rutina de interrupción del timer (`ontimerISR`). Se calcula la distancia recorrida transformando los "ticks" o cuentas del encoder a metros, utilizando la relación geométrica de la rueda:

$$\text{Distancia} = \frac{\text{Cuentas Actuales}}{\text{Cuentas por Vuelta}} \times 2 \cdot \pi \cdot \text{Radio Rueda} \quad (1)$$

En el código, esto se traduce de la siguiente manera para determinar cuándo detener el robot:

```
1 // Cálculo de distancia en la interrupción
2 // Usamos abs() para asegurar que la distancia siempre suma
3 double vueltas_actuales = (double)abs(contador_m1) / (double)
  vueltas;
4 double distancia = vueltas_actuales * 2.0 * M_PI * radio_rueda;
5
6 // Condición de parada:
7 if (distancia >= DISTANCIA_OBJETIVO){
8     referencia_m1 = 0;
9     referencia_m2 = 0;
10    objetivo_alcanzado = true;
11 }
```

(Nota: se han añadido signos negativos al cálculo de los contadores puesto que si no se invertía, al dar una velocidad positiva el robot iba hacia atrás).

1.3. Metodología de Calibración Experimental

A pesar de tener un modelo teórico, las imperfecciones físicas (compresión de la goma de la rueda, deslizamiento imperceptible) causan discrepancias entre la distancia calculada y la real. Para solucionar esto, se siguió el siguiente procedimiento experimental:

1. **Medición Inicial:** Se midió el radio físico de la rueda y se introdujo como parámetro inicial.
2. **Prueba de Campo:** Se ejecutó el programa solicitando un avance de 2.0 metros.
3. **Verificación:** Se utilizó una cinta métrica en el laboratorio para medir el desplazamiento real del robot.
4. **Ajuste:**
 - Si el robot recorría menos de 2 metros, significaba que el software "pensaba" que la rueda era más grande de lo que realmente actuaba.
 - Si el robot recorría más de 2 metros, el radio configurado era demasiado pequeño.
5. **Resultado:** Se ajustó iterativamente la variable `radio_rueda` hasta que la distancia de parada del robot coincidió exactamente con la marca de 2 metros de la cinta métrica.

El valor final calibrado que compensa estos errores sistemáticos quedó definido en el código como:

```
1 const float radio_rueda = 0.0335;
```

Una vez se obtuvo el radio final, se verificó la condición de velocidad; el robot era capaz de recorrer 0.1 metros en un segundo, confirmando que el radio era correcto (ya que el robot cumplía satisfactoriamente el control PID de velocidad en radianes).

1.4. Autoevaluación

Puesto que el robot es capaz de avanzar en línea estrictamente recta sin desviarse demasiado durante varios metros y alcanza las velocidades del apartado, considero mi calificación para esta tarea de un 9,5/10. Ya que al recorrer unos 8 metros, el robot se desvía $12,4^{\circ}$, pues no es perfecto.

2. Trazar una circunferencia de 0.4 m de radio y velocidad angular $w = \frac{2\pi}{10}$ rad/s.

2.1. Planteamiento del Problema y Cálculos Cinemáticos

El objetivo de esta tarea consistió en programar el robot para describir una trayectoria circular perfecta con un radio $R = 0,4$ metros, manteniendo una velocidad angular constante de $w = \frac{2\pi}{10}$ rad/s durante un tiempo $t = 10$ segundos.

Para lograr este movimiento en un robot de tracción diferencial, es necesario calcular la relación entre la velocidad angular de giro del chasis (w) y la velocidad lineal de avance (v). Basándonos en la cinemática del movimiento circular uniforme:

$$v = w \cdot R \quad (2)$$

Sustituyendo los valores dados:

$$v = \left(\frac{2\pi}{10} \text{ rad/s} \right) \cdot 0,4 \text{ m} \quad (3)$$

Por lo tanto, el centro del chasis debe moverse a 0,251 m/s mientras rota sobre sí mismo a 0,628 rad/s. O más simplemente en Arduino:

```
1 // --- CONTROL DE NAVEGACION ---  
2 const float vel_ang = 0.2 * M_PI;  
3 const float vel_lin = vel_ang * 0.4;
```

2.2. Implementación en el Código de Control

En el firmware, se definieron estas consignas cinemáticas como constantes globales. El código utiliza un modelo de cinemática inversa para transformar estas velocidades deseadas (w) en velocidades angulares individuales para cada rueda.

El cálculo de las velocidades de referencia para los PID se realiza mediante las siguientes ecuaciones, donde L es la distancia entre ruedas (0.21 m) y R el radio de la rueda (0.0335 m):

$$w_{der} = \frac{v + w(L/2)}{R} \quad ; \quad w_{izq} = \frac{v - w(L/2)}{R} \quad (4)$$

En el código (loop principal), esto se traduce así:

```
1 float v_rad_der = -(vel_lin + vel_ang * distancia_ruedas / 2.0) /  
    radio_rueda;  
2 float v_rad_izq = -(vel_lin - vel_ang * distancia_ruedas / 2.0) /  
    radio_rueda;
```

(Nota: Los signos negativos se deben a la orientación física de los motores).

2.3. Control de Duración de la Trayectoria

Dado que el enunciado especifica una duración exacta de 10 segundos, el robot completará exactamente una vuelta completa (2π radianes). Para detener el robot en el punto exacto, se optó por un control basado en odometría de arco. Se calculó la distancia recorrida por la rueda exterior:

$$D_{motor_{exterior}} = 2 \cdot \pi \cdot (R + L/2) = 3,12 \quad (5)$$

En el código, se estableció una condición de parada basada en la distancia media recorrida por los encoders. Para simplificar, se calculó la distancia que debía recorrer la rueda exterior como condicion de parada:

```
1 // Definimos la distancia objetivo AQU para usarla en todo el
  c digo
2 const double DISTANCIA_OBJETIVO = 3.12;
3 bool objetivo_alcanzado = false;
4
5 double vueltas_actuales = (double)abs(contador_m1) / (double)
  vueltas;
6 double distancia = vueltas_actuales * 2.0 * M_PI * radio_rueda;
7
8 // Lógica de parada en el Timer (Interrupción)
9 // Si llegamos a la distancia, ponemos referencias a 0 y
  activamos la bandera
10 if (distancia >= DISTANCIA_OBJETIVO){
11     referencia_m1 = 0;
12     referencia_m2 = 0;
13     objetivo_alcanzado = true;
14 }
```

DISTANCIA_OBJETIVO a 3.12 metros (valor empírico).

2.4. Autoevaluación

Puesto que el robot es capaz de realizar la circunferencia de radio perfectamente 0.4 m sin desviarse durante , considero mi calificación para esta tarea de un 9,5/10. Como pega diré que al terminar se pasa unos milímetros antes de pararse.

3. Recorrer 2 metros en línea recta con control de orientación.

A partir de este enunciado ya he incluido ROS2, publicando la pose del robot en el topic `robot_pose`, pero lo explicaré en la tarea 7.

3.1. Estrategia de Control: Odometría Diferencial

A diferencia de los apartados anteriores, en esta etapa se implementó un sistema de Odometría Diferencial. Esto permite al robot conocer su Pose (x, y, θ) en todo momento para verificar la rectitud de la trayectoria. El cálculo se realiza dentro de la rutina de interrupción del timer `ontimerISR` cada 10 ms ($T_s = 0,01s$), lo que garantiza una alta resolución temporal en la integración de la posición.

3.2. Implementación Matemática del Modelo

Para estimar el estado, primero se transforman las velocidades angulares de las ruedas a velocidades del chasis:

$$v_{robot} = \frac{v_{der} + v_{izq}}{2} \quad ; \quad w_{robot} = \frac{v_{der} - v_{izq}}{L} \quad (6)$$

Donde L es la distancia entre ruedas. Posteriormente, se integran estas velocidades usando una aproximación de punto medio (Runge-Kutta orden 2 simple) para reducir el error de deriva:

```
1 // 1. Cálculo del incremento de orientación
2 double delta_theta = w_robot * tiempo_muestreo;
3 theta -= delta_theta; // Actualización del rumbo acumulado
4
5 // Normalización del ángulo entre -PI y PI
6 if (theta > M_PI) theta -= 2.0 * M_PI;
7 if (theta < -M_PI) theta += 2.0 * M_PI;
8
9 // 2. Proyección y acumulación de coordenadas (X, Y)
10 // Se usa el ángulo medio para mayor precisión
11 pos_x -= v_robot * cos(theta - (delta_theta / 2.0)) *
    tiempo_muestreo;
12 pos_y -= v_robot * sin(theta - (delta_theta / 2.0)) *
    tiempo_muestreo;
```

3.3. Mantenimiento del Rumbo (Control en Lazo Cerrado)

El requisito de “Control de Orientación” se satisface mediante la acción conjunta de la Cinemática Inversa y los Controladores PID.

1. **Generación de Referencia:** Al solicitar movimiento recto, se inyecta $w = 0$.
2. **Corrección de Perturbaciones:** Si una rueda se frena, el PID detecta el error y aumenta el PWM para igualar a la otra rueda, manteniendo $w \approx 0$ y $\theta \approx cte$.



3.4. Autoevaluación

Puesto que el robot es capaz de avanzar en línea estrictamente recta sin desviarse demasiado durante varios metros y alcanza las velocidades del apartado, considero mi calificación para esta tarea de un 10/10.

4. Implementar un algoritmo de seguimiento de trayectoria (Follow the carrot)

En este apartado se describe la implementación de un sistema de navegación autónoma por puntos de paso (*waypoints*). El objetivo es que el robot recorra una secuencia predefinida de coordenadas cartesianas (x, y) , simulando un recorrido por distintas estancias (pasillo, salón, cocina), basándose exclusivamente en la odometría calculada en el apartado anterior.

4.1. Definición de la Trayectoria

Para gestionar la ruta, se definió una estructura de datos `Point` y un vector de coordenadas que representan los hitos del camino. El sistema utiliza un índice `indice_punto_actual` para gestionar secuencialmente el objetivo a alcanzar.

```
1 struct Point { float x; float y; };
2
3 Point trayectoria[5] = {
4     {1.0, 0.0}, // Punto 1: Salida al pasillo
5     {1.0, 8.0}, // Punto 2: Recorrer pasillo principal
6     {0.0, 8.0}, // Punto 3: Entrada a zona salon
7     {0.0, 9.0}, // Punto 4: Aproximacion cocina
8     {2.0, 9.0}  // Punto 5: Destino final
9 };
```

Listing 1: Estructura de Waypoints

4.2. Algoritmo de Control: Navegación Punto a Punto

El núcleo del sistema de navegación se ejecuta en el bucle principal y se basa en el cálculo vectorial del error de posición respecto al objetivo activo. Para cada ciclo de control, se calculan dos parámetros fundamentales:

1. **Distancia al objetivo (d):** Distancia euclídea entre la posición actual estimada por la odometría (x, y) y el punto destino (x_{ref}, y_{ref}) .

$$d = \sqrt{(x_{ref} - x)^2 + (y_{ref} - y)^2} \quad (7)$$

2. **Error Angular (θ_{err}):** Diferencia entre el ángulo necesario para mirar al objetivo (bearing) y la orientación actual del robot (θ).

$$\theta_{goal} = \text{atan2}(y_{ref} - y, x_{ref} - x) \quad (8)$$

$$\theta_{err} = \theta_{goal} - \theta \quad (9)$$

Es crítico normalizar el error angular dentro del rango $[-\pi, \pi]$ para asegurar que el robot siempre elija el giro más corto (por ejemplo, girar -10° en lugar de $+350^\circ$).

4.3. Máquina de Estados de Navegación

Para asegurar un movimiento suave y preciso, se implementó una lógica de control híbrida que prioriza la orientación sobre el avance. El algoritmo evalúa las condiciones en el siguiente orden:

1. Verificación de llegada: Si la distancia al objetivo es menor que la tolerancia establecida ($d < 0,03$ m), el sistema considera el punto alcanzado, incrementa el índice del vector y selecciona el siguiente objetivo.

2. Corrección de Orientación (Giro Puro): Si el error angular es elevado ($|\theta_{err}| > 0,35$ rad), el robot detiene su avance lineal ($v = 0$) y aplica una velocidad angular proporcional al error para encarar el objetivo rápidamente.

$$\omega = K_p \cdot \theta_{err} \quad (\text{donde } K_p = 2,0)$$

3. Avance con Corrección (Seguimiento): Una vez que el robot está razonablemente alineado con el objetivo, se activa la velocidad lineal de cruceo ($v = 0,2$ m/s) manteniendo una corrección angular suave para compensar desviaciones durante el trayecto.

```
1 // Calculo de errores
2 float distancia_obj = sqrt(dx*dx + dy*dy);
3 float angulo_obj = atan2(dy, dx);
4 float error_theta = angulo_obj - curr_th;
5
6 // Normalizacion del angulo
7 while (error_theta > M_PI) error_theta -= 2.0 * M_PI;
8 while (error_theta < -M_PI) error_theta += 2.0 * M_PI;
9
10 if (distancia_obj < DISTANCIA_TOLERANCIA) {
11     indice_punto_actual++; // Siguiente punto
12 }
13 else {
14     if (abs(error_theta) > 0.35) {
15         // Giro estatico si el error es grande
16         vel_lin = 0.0;
17         vel_ang = 2.0 * error_theta;
18     } else {
19         // Avance con correccion suave
20         vel_lin = 0.2;
21         vel_ang = 1.0 * error_theta;
22     }
23 }
```

Listing 2: Lógica de Control de Navegación

Este enfoque desacoplado permite que el robot navegue complejas trayectorias sin necesidad de planificadores de ruta avanzados, confiando en la robustez de los lazos de control PID de bajo nivel para ejecutar las referencias de velocidad generadas.

El recorrido de la Figura 1 adjunta puede observarse en el vídeo del proyecto.



Figura 1: Trayectoria para ir de mi habitación a la cocina.

4.4. Autoevaluación

El robot es capaz de ir desde mi habitación a la cocina correctamente, que están a una distancia considerablemente muy lejana, por lo que considero mi nota en esta tarea de 10/10.

5. Sistema de Seguridad Reactiva mediante Sensores de Proximidad

5.1. Integración y Adquisición de Datos

Para dotar al robot de capacidad de percepción del entorno y evitar colisiones, se han integrado tres sensores de ultrasonidos (HC-SR04) dispuestos en configuración frontal, lateral izquierda y lateral derecha. Para medirlos, se implementó una función específica *leerSonar* que optimiza este proceso para no afectar al rendimiento del microcontrolador.

Timeout de Seguridad: La función estándar *pulseIn* puede bloquear el procesador hasta 1 segundo si no recibe eco. Se implementó un timeout de 25.000 microsegundos ($\approx 25\text{ms}$), limitando el rango de detección a unos 4 metros. Si el sensor no recibe respuesta en este tiempo (por ejemplo, cable desconectado o espacio abierto), la función asume camino libre (300 cm) en lugar de bloquear el bucle de control.

```
1 long duracion = pulseIn(echoPin, HIGH, 25000);  
2 if (duracion == 0) return 300.0;
```

5.2. Estrategia de Lectura Secuencial

La lectura de ultrasonidos es un proceso lento comparado con la frecuencia de cálculo del PID (100 Hz). Leer los tres sensores consecutivamente en cada ciclo del loop introduciría retardos inaceptables.

Para solucionar esto, se implementó un planificador de tareas no bloqueante basado en *millis()*. Los sensores se leen solo una vez cada 100 ms. Además, se introdujeron pequeños retardos de 10 ms entre la lectura de cada sensor para disipar los ecos residuales y evitar el fenómeno de interferencia acústica entre sensores vecinos.

```
1 if (millis() - last_sonar_read > 100) {  
2     last_sonar_read = millis();  
3     dist_frontal = leerSonar(pinTrig1, pinEcho1);  
4     delay(10);  
5     dist_izq = leerSonar(pinTrig2, pinEcho2);  
6     delay(10);  
7     dist_der = leerSonar(pinTrig3, pinEcho3);  
8 }
```

5.3. Lógica de Seguridad (Override de Control)

La lógica de parada sobrescribe las referencias de velocidad antes de llegar a los PID.

- **Caso A (Camino Libre):** Si $d > 15$ cm, se permite el paso de las velocidades calculadas.
- **Caso B (Obstáculo Detectado):** Si cualquiera de los sensores detecta un objeto a menos de 15 cm, se fuerza la referencia a 0.

```
1 // Mantenemos la seguridad de los sonares
2 if(dist_der>15 and dist_izq>15 and dist_frontal>15 ){
3     referencia_m1 = v_rad_der * (vueltas / (2.0 * M_PI));
4     referencia_m2 = v_rad_izq * (vueltas / (2.0 * M_PI));
5 } else {
6     referencia_m1=0;
7     referencia_m2=0;
8 }
```

5.4. Autoevaluación

El robot es capaz de usar correctamente los sónares, pero a veces fallan, imagino que por la mala calidad del mismo, y si el robot persiste durante mucho rato, es capaz de chocarse contra un objeto si falla el sonar momentaneamente 8,5/10.

6. Implementación de la Tarea 5 con micro-ROS

En este apartado se evoluciona el sistema hacia una arquitectura distribuida mediante el uso de **micro-ROS**. El objetivo es externalizar la lógica de navegación de alto nivel a un nodo de ROS 2 ejecutado en el ordenador, dejando al ESP32 como un controlador de bajo nivel encargado de la odometría y el control de motores.

6.1. Arquitectura del Sistema Distribuido

La comunicación se establece mediante transporte WiFi (UDP). El sistema se divide en dos nodos principales que intercambian información de forma asíncrona:

- **ESP32 (Low-Level Driver):** Calcula la posición (x, y, θ) y ejecuta los lazos de control PID de velocidad.
- **PC (High-Level Controller):** Contiene el planificador de trayectoria y el algoritmo *Follow the Carrot*.

6.2. Configuración del Firmware ESP32

El microcontrolador actúa ahora como un puente entre el hardware y el middleware de ROS 2. Se han configurado dos interfaces de comunicación principales:

1. Publicación de Estado (/robot_pose): El ESP32 calcula la odometría en la interrupción del timer (como se explicó en el apartado 3) y publica el resultado a la red para que el PC sepa dónde está el robot.

```
1 // Se ejecuta a 20Hz en el loop principal
2 pose_msg.x = pos_x;
3 pose_msg.y = pos_y;
4 pose_msg.theta = theta;
5 rcl_publish(&pose_publisher, &pose_msg, NULL);
```

Listing 3: Publicación de Odometría en ESP32

2. Suscripción de Comandos (/cmd_vel): El robot se suscribe al tópico estándar de velocidad. Cuando el nodo de ROS 2 envía una orden, el *callback* actualiza las referencias globales que alimentan la cinemática inversa.

```
1 void subscription_callback(const void * msgin) {
2     const geometry_msgs__msg__Twist * msg = (const
3         geometry_msgs__msg__Twist *)msgin;
4     // Las referencias de velocidad vienen externamente
5     vel_lin = msg->linear.x;
6     vel_ang = msg->angular.z;
7 }
```

Listing 4: Callback de Suscripción

Es importante notar que el ESP32 mantiene una **capa de seguridad reactiva local**: aunque recibe órdenes del PC, si los sensores de ultrasonidos detectan un obstáculo cercano (< 15 cm), el firmware anula la orden externa y detiene los motores para evitar daños.

6.3. Nodo de Control en ROS 2: Follow Carrot

La lógica de seguimiento de trayectoria se ha migrado a un nodo de C++ (`follow_carrot_node`). Este nodo define la lista de *waypoints* y cierra el lazo de control de posición.

Definición de Trayectoria: A diferencia de la implementación anterior, los puntos se definen en la memoria del PC, permitiendo trayectorias más complejas o dinámicas sin reprogramar el microcontrolador.

```
1 trajectory_ = {  
2     {0.8, 0.0}, {0.8, 2.0}, {0.8, 4.0}, {0.8, 6.0}, {0.8, 8.0}  
3 };
```

Listing 5: Definición de Trayectoria en ROS 2

Algoritmo de Navegación: El nodo recibe la pose del ESP32 y calcula el comando de velocidad necesario. Se ha implementado una máquina de estados para priorizar el giro sobre el avance:

1. Se calcula el error angular hacia el objetivo (θ_{err}).
2. Si $|\theta_{err}| > 0,08$ rad ($\approx 4,5^\circ$), el nodo envía velocidad lineal 0 y velocidad angular proporcional al error (giro estático).
3. Si el robot está alineado, se envía una velocidad lineal progresiva (con rampa de aceleración) y correcciones angulares suaves.

```
1 // Si el error angular es grande, solo giramos  
2 if (std::abs(yaw_error) > 0.08) {  
3     cmd_msg.linear.x = 0.0;  
4     cmd_msg.angular.z = std::min(kp * yaw_error, max_ang);  
5 } else {  
6     // Si estamos alineados, avanzamos con aceleracion suave  
7     cmd_msg.linear.x = std::min(vel_ant + 0.1, max_lin);  
8 }  
9 pub_cmd_ ->publish(cmd_msg);
```

Listing 6: Lógica de Control en el Timer del Nodo

Esta arquitectura permite monitorizar en tiempo real el error de seguimiento desde la terminal del PC y facilita la integración futura con herramientas de visualización como Rviz.

6.4. Autoevaluación

He conseguido implementar a la perfección ROS2 y el algoritmo follow the carrot. EL único problema que hay es que la conexión entre el ordenador y el robot usando ros2 es lenta y genera muchos retrasos, haciendo que el ángulo no se corrija correctamente y tenga que volver a corregir. Aunque considero que es propio de la tecnología y no es mi culpa, consideraré la nota un 9/10.

7. Tareas Opcionales

7.1. Construcción, Diseño y Mejoras Electromecánicas

Se ha realizado un diseño personalizado bajo la temática de *Tiburón Azul*, combinando una base de madera (resistencia) con una carcasa de cartón pluma (ligereza) y cierre magnético. Además del acabado estético, se han implementado tres mejoras funcionales clave:

- **Freno de Estacionamiento Analógico:** El sistema de doble interruptor se ha diseñado mecánicamente para que, al apagar el robot, la propia palanca bloquee físicamente el neumático, inmovilizando el vehículo.
- **Monitorización y Carga:** Se ha integrado un voltímetro digital en el chasis para control de baterías y un puerto de carga USB-C accesible desde el exterior.
- **Interruptores Independientes:** Permiten separar la alimentación de potencia (motores) de la lógica, facilitando la depuración sin riesgo de movimiento.



Figura 2: Diseño final del robot con temática de tiburón.



Figura 3: Diseño final del robot con voltímetro integrado.

7.2. Implementación de Algoritmo de Seguimiento "FollowMe" mediante Sonar

7.2.1. Descripción del Procedimiento

Se ha añadido una nueva funcionalidad al sistema de navegación del robot denominada *Follow Me*. El objetivo es mantener una distancia constante (d_{target}) respecto al objeto situado inmediatamente delante del robot.

El algoritmo se basa en un controlador proporcional (P) simple que opera dentro del bucle de control principal:

1. **Lectura:** El sensor ultrasónico frontal mide la distancia actual (d_{actual}).
2. **Cálculo del Error:** Se calcula la diferencia respecto al punto de referencia establecido en 30 cm:

$$e(t) = d_{actual} - 30 \text{ cm}$$

3. **Acción de Control:** La velocidad lineal (v) se ajusta proporcionalmente al error:

$$v = K_p \cdot e(t)$$

Si $e(t) > 0$ (robot lejos), v es positiva (avanza). Si $e(t) < 0$ (robot cerca), v es negativa (retrocede). Se ha implementado una zona muerta (± 5 cm) para evitar oscilaciones en estado estacionario.

7.2.2. Resultados y Conclusiones

Tras las pruebas experimentales, se concluye que el rendimiento del seguimiento es **deficiente**. Aunque la lógica de control es correcta, la tecnología de ultrasonidos presenta limitaciones físicas críticas para el seguimiento de seres humanos:

- **Pérdida de Ecos (Reflexión Especular):** La ropa y las formas irregulares del cuerpo humano (piernas al caminar) actúan como superficies absorbentes o planos inclinados. [Image of ultrasonic sensor specular reflection diagram] Las ondas sonoras rebotan en direcciones aleatorias y no regresan al receptor, provocando lecturas falsas de "espacio vacío".
- **Ruido en Movimiento:** La vibración propia del robot al desplazarse introduce ruido en la señal del eco, perdiendo precisión en la medición continua.
- **Confusión de Objetivos:** El haz del sonar es cónico y amplio. El robot es incapaz de discriminar entre la persona a seguir y otros obstáculos estáticos cercanos (paredes, mesas), perdiendo frecuentemente el objetivo ("target lock").

7.2.3. Propuesta de Mejora

Para lograr un seguimiento robusto y fiable, se propone la sustitución de la sensórica actual por visión artificial mediante una **ESP32-CAM**. [Image of ESP32-CAM module]

El uso de una cámara permitiría implementar algoritmos de seguimiento visual (como detección de color *Blob Tracking* o detección facial básica), eliminando los problemas de rebotes acústicos y permitiendo diferenciar al usuario del entorno estático.

7.3. Interfaz Web de Teleoperación y Control Unificado

En esta fase del proyecto se ha dado un salto cualitativo en la complejidad del desarrollo. Para materializar funcionalidades avanzadas de control, se ha combinado el **criterio ingenieril** adquirido con la potencia de la **Inteligencia Artificial Generativa**. Esta sinergia ha permitido implementar arquitecturas de software complejas (Servidores Asíncronos, APIs de navegador, WebSockets) para transformar las ideas conceptuales en código funcional de manera eficiente.

Fruto de este enfoque, se ha desarrollado una interfaz web (SPA) alojada en el ESP32 que actúa como un **Centro de Mando Integral**. El sistema no solo permite la teleoperación, sino que unifica todas las tareas autónomas desarrolladas durante el curso en un solo panel de control accesible vía WiFi.

7.3.1. Modos de Control y Funcionalidades

La interfaz se comunica mediante **WebSockets** para garantizar baja latencia y permite la ejecución de los siguientes modos de operación:

1. Control Manual (Teleoperación):

- *Joystick Virtual*: Implementado con HTML5 Canvas para control táctil desde dispositivos móviles.
- *Mando PS4 (Gamepad API)*: Se utiliza la API del navegador para capturar los inputs de un mando DualShock 4 conectado por Bluetooth al cliente (tablet/PC) y enviarlos al robot por WiFi. Esto descarga al ESP32 de la gestión del stack Bluetooth.

Complementariamente se ha añadido un botón de parada de emergencia, que detiene en seco el robot en caso de que el operario lo desee.

2. Gestión de Tareas Autónomas (Parametrizables):

A diferencia de las prácticas anteriores donde los parámetros eran fijos en el código, la interfaz web permite configurar y lanzar las misiones en tiempo real:

- *Ir a Punto (Línea Recta)*: Campos de entrada para enviar coordenadas destino (X, Y). El robot calcula el giro y avance necesarios.
- *Trayectoria Circular*: Permite definir el radio R (en metros) desde la web e iniciar la maniobra cinemática.
- *Follow the Carrot (Waypoints)*: Se ha implementado un editor de rutas en la interfaz que permite añadir una lista dinámica de puntos. Al pulsar **Ejecutar**, el array completo de coordenadas se envía al robot para que realice el seguimiento de trayectoria secuencial.

```
1 // JavaScript en el cliente: Mapeo de ejes y zona muerta
2 function updateGamepad() {
3     var gp = navigator.getGamepads()[0];
4     if (gp) {
5         var yRaw = Math.abs(gp.axes[1]) > 0.1 ? gp.axes[1] : 0;
6         // Deadzone
7         var xRaw = Math.abs(gp.axes[0]) > 0.1 ? gp.axes[0] : 0;
8
9         // Modo Turbo con R1
10        var currentMax = gp.buttons[5].pressed ? 0.6 : maxVel;
11
12        // Envío de comandos al ESP32
13        ws.send('V,$((-yRaw * currentMax).toFixed(2)),$((-xRaw *
14            1.5).toFixed(2))');
```

Listing 7: Lectura del Gamepad y envío por WebSocket

Esta implementación convierte al robot en una plataforma versátil donde es posible alternar entre control manual y misiones autónomas complejas sin necesidad de reprogramar el firmware.

7.3.2. Visualización Topológica, Mapeo SLAM y Navegación Táctil

Como característica distintiva de esta interfaz, se ha desarrollado un sistema de visualización en tiempo real utilizando la API **Canvas** de HTML5. Este módulo permite al operador no solo ver datos numéricos, sino interpretar espacialmente el comportamiento del robot, habilitando el control sin línea de visión directa.

1. Panel de Telemetría y Sensores: En la parte superior de la interfaz se ha dispuesto un panel de datos (*Dashboard*) que muestra, en tiempo real, el valor numérico exacto en centímetros medido por cada uno de los tres sensores de ultrasonidos (Frontal, Izquierdo y Derecho). Esto permite al operador contrastar la precisión del mapa visual con la lectura cruda del sensor, facilitando la depuración y la navegación de precisión en espacios estrechos.

2. Representación Odometrica en Tiempo Real: El sistema dibuja un “avatar” del robot y su estela (trayectoria histórica) sobre un lienzo negro. Para facilitar la orientación subjetiva del piloto en la pantalla del móvil, se ha realizado una transformación de coordenadas donde el eje X positivo representa el avance vertical (hacia arriba) y el eje Y positivo el desplazamiento lateral izquierdo.

3. Algoritmo de Mapeo por Sonar (Occupancy Grid): Dada la ausencia de sensores láser (LiDAR), se ha ideado una **solución ingeniosa** para realizar un mapeo del entorno utilizando los tres sensores de ultrasonidos HC-SR04. El algoritmo proyecta las mediciones numéricas mostradas en el panel al sistema de coordenadas global, pintando “puntos verdes.” en el mapa:

1. Se lee la distancia d de cada sensor.
2. Se aplica una matriz de rotación basada en la orientación actual del robot (θ) y el ángulo de montaje del sensor (ϕ_{sensor}).

3. Se calcula la posición absoluta del obstáculo:

$$P_{obs_x} = P_{rob_x} + d \cdot \cos(\theta + \phi_{sensor})$$

$$P_{obs_y} = P_{rob_y} + d \cdot \sin(\theta + \phi_{sensor})$$

Análisis Crítico de la Tecnología: Es fundamental reseñar que esta implementación es una versión "débil" de SLAM (*Simultaneous Localization and Mapping*). Debido a la baja calidad de los sonares (cono de detección amplio y rebotes especulares), el mapa generado presenta ruido. Sin embargo, la arquitectura del software es escalable: sustituyendo únicamente los datos de entrada de los sonares por un **barrido láser (LiDAR)**, la calidad del mapa mejoraría exponencialmente, convirtiéndose en una solución industrial sin modificar el código de visualización.

4. Modo de Seguimiento Táctil ("Finger Follow"): Aprovechando la generación del mapa, se ha implementado un modo de control de alto nivel. Al tocar o hacer clic en cualquier punto del mapa (zona negra o inexplorada), el sistema:

1. Traduce las coordenadas de píxeles de la pantalla a coordenadas del mundo real (metros).
2. Envía un comando de navegación G (Go-To) al robot.
3. El robot se orienta y avanza autónomamente hacia la posición del dedo.

Esta funcionalidad permite controlar el robot desde otra habitación, guiándolo únicamente a través del mapa que él mismo va generando en tiempo real.

```
1 function updateGrid(rx, ry, rth, dist_m, sensorOffset) {  
2   // Proyeccion de coordenadas locales a globales  
3   var angulo_global = rth + sensorOffset.a;  
4   var obs_x = rx + dist_m * Math.cos(angulo_global);  
5   var obs_y = ry + dist_m * Math.sin(angulo_global);  
6  
7   // Incremento de probabilidad en la celda (Punto Verde)  
8   var key = Math.round(obs_x/gSize) + "," + Math.round(obs_y/  
9     gSize);  
10  if(grid[key] === undefined) grid[key] = 0;  
11  grid[key] += 3; // Aumentar intensidad visual  
}
```

Listing 8: Algoritmo JS de proyección de obstáculos en el mapa

7.4. Autoevaluación

Como tareas opcionales, considero que he introducido muchas funciones y muy innovadoras como el mapa y el seguimiento. Además, he obtenido conclusiones y posibles mejoras de las novedades incluidas, como el uso de barrido laser para creación del mapa y de seguimiento. Por ello, consideraré mi nota en tareas extra de 10/10.

8. Video donde ver las tareas

ENLACE PARA VER EL VIDEO DE LAS TAREAS.

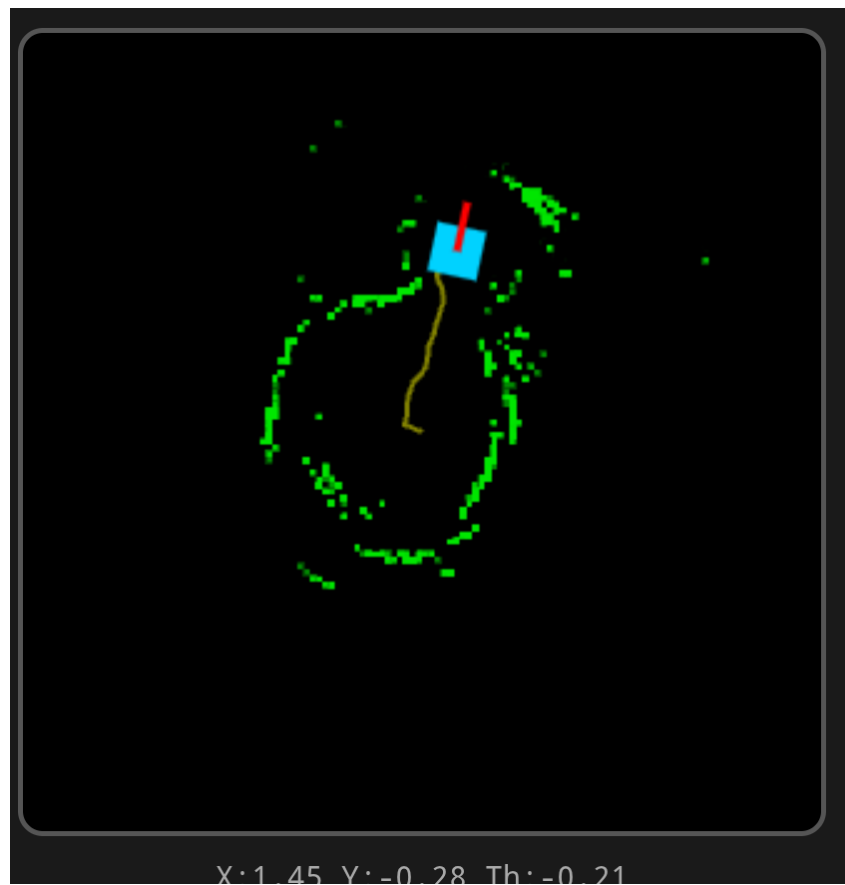


Figura 4: Mapeo de mi habitación y parte del pasillo.

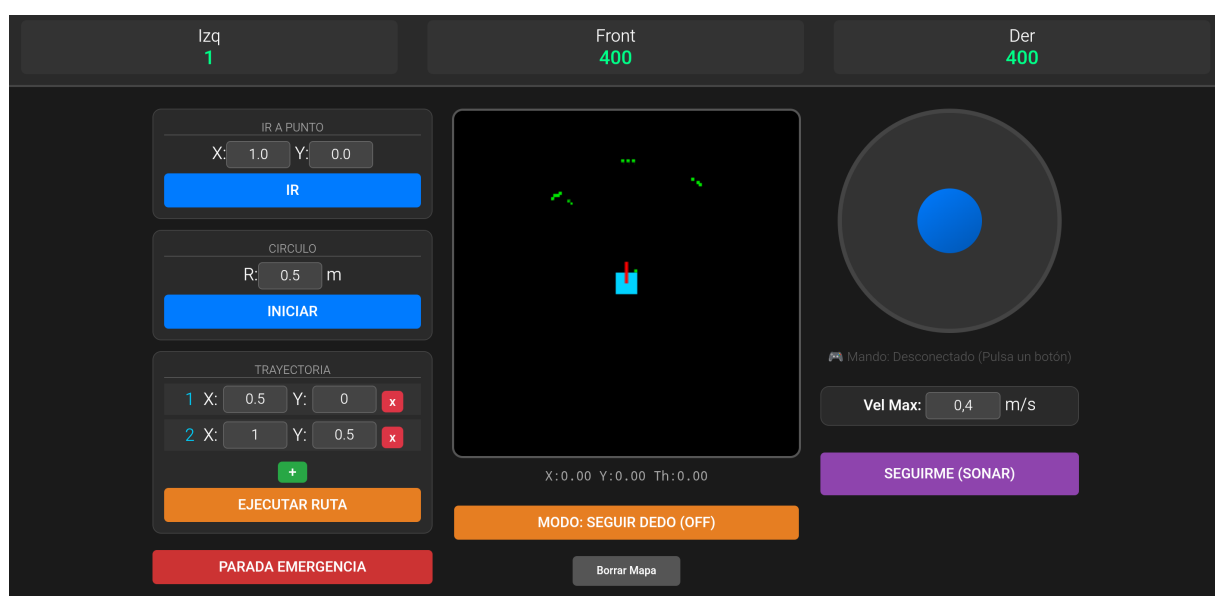


Figura 5: Diseño final de la interfaz web del robot.